

Gdańsk University of Technology
Faculty of Electronics, Telecommunications and Informatics
Department of Multimedia Systems

Technical Documentation
Mobile Application & Backend
v1.0

**System Architecture and Application
Technologies**

Project Subject: Glaucoma detection using intelligent
analysis of fundus images

Team Members: Martyna Borkowska (Lead), Karolina Glaza,
Mateusz Dobry, Agnieszka Pawłowska,
Amila Amarasekara, Antoni Naczke

Supervisor: Prof. Andrzej Czyżewski, Ph.D., D.Sc., Eng.

Authors: Karolina Glaza

March, 2026

Contents

1	Architecture	2
2	Mobile Application	2
2.1	Swift	2
3	Backend	2
3.1	FastAPI	2
4	Artificial Intelligence Module	3
5	Data Flow	3
6	Summary	3

1 Architecture

The system consists of three main components: a mobile application, a backend, and an image-analysis module based on AI models. The components communicate over an HTTP-based API.

2 Mobile Application

The application is written in Swift using the SwiftUI framework. Its role is to let the user capture or select a fundus photograph, send it to the backend for analysis, and present the returned results.

2.1 Swift

A modern programming language developed by Apple for building applications for iOS, iPadOS, macOS, and other platforms of the Apple ecosystem. It enables native applications that fully exploit the capabilities of these devices.

Advantages

- Native iOS language — the app runs smoothly on every iPhone model without extra responsiveness tuning.
- Full compliance with iOS standards and direct access to native device features (camera).
- No additional intermediate layers between the application code and the operating system.
- Possibility of integrating with the Apple ecosystem, e.g. Apple Health (heart rate, physical activity, other data).
- Closer to a production-grade solution — an official Apple technology used in real apps.

Disadvantages

- A Mac computer is required for the full development and debugging workflow.
- No native cross-platform support — the app runs only within the Apple ecosystem.
- Limited code reuse if versions for other platforms (e.g. Android) are planned.

3 Backend

The backend is written in Python using the FastAPI framework. It receives requests from the application, processes the uploaded images, and communicates with the AI module. It exposes an API endpoint for uploading images and returning results.

3.1 FastAPI

A lightweight Python framework for quickly building HTTP APIs that the Swift mobile application can communicate with.

Advantages

- Natural integration with Python, in which the AI models (YOLO, UNet++) are implemented.
- High performance (asynchronous processing).
- Easy communication with the iOS app through standard HTTP requests.

Disadvantages

- Requires running a separate backend server.
- Requires managing infrastructure (hosting, networking, API security).

4 Artificial Intelligence Module

Fundus image analysis is performed by a pipeline of two machine-learning models:

- **YOLO** — detection of the region of interest (ROI) in the fundus image.
- **UNet++** — segmentation of structures relevant to glaucoma diagnosis.

The image is first processed by YOLO to localise the region containing the optic disc. The extracted fragment is then passed to UNet++, which performs segmentation and enables further analysis of glaucoma-characteristic changes.

5 Data Flow

The system operates in the following steps:

1. The user selects a fundus photograph from the gallery in the mobile application.
2. The image is sent to the backend via an HTTP request.
3. The backend forwards the image to the AI model pipeline.
4. The YOLO model detects the region of interest (ROI).
5. The UNet++ model segments the selected region.
6. The analysis result is returned to the mobile application.
7. The application presents the result to the user.

This data flow is designed for the first version of the application (gallery images). Future versions plan to acquire data either from the user's camera or from professional equipment available at the clinic. A short in-app medical questionnaire is also planned, which could additionally support the model.

6 Summary

The proposed architecture cleanly separates the three concerns — presentation (Swift/SwiftUI), orchestration (FastAPI backend), and analysis (YOLO + UNet++ pipeline) — connected over a simple HTTP API. This keeps each component independently replaceable and provides a clear path toward later versions (camera capture, professional equipment, medical questionnaire).